

A



AIML Lab sheet 5

IT25102378

Exercise 2

AI Model 1: DeepSeek

Current ranking: 23

deepseek

🔍

📄

Apple M chip architecture
⚡ Instant

New chat

Today

Apple M chip architecture

Explain the apple M chip architecture

📄

✍️

To explain the Apple M-series chip architecture, you have to throw out much of what you know about traditional PC processors (like Intel or AMD).

The M-series is not just a CPU; it is a **System on a Chip (SoC)** built around a philosophy of **unified memory**, **massive efficiency**, and **specialized compute engines**. It is an Arm-based architecture, but Apple has customized it so heavily that it exists in a league of its own.

Here is a breakdown of the M-chip architecture, from its foundational design to its individual components.

1. The Foundation: Arm (But Not Stock Arm)

While Intel uses x86 and AMD uses x86-64, Apple uses the **Arm architecture** (specifically, Armv8 and now Armv9 extensions). However, Apple does not use off-the-shelf Arm cores (like Cortex). They purchase an **Architecture License**, meaning they build their own custom CPU cores from the ground up using the Arm instruction set.

- **The Result:** Apple's custom cores (code-named *Firestorm*, *Icestorm*, *Avalanche*, *Blizzard*, etc.) are incredibly wide. They can decode, rename, and execute significantly more instructions per clock cycle (IPC) than comparable x86 chips, giving them massive single-core performance at very low clock speeds (usually around 3.2–4.0 GHz, compared to Intel's 5.0+ GHz).

2. The "Hybrid" CPU: Performance & Efficiency Cores

All M-series chips use a heterogeneous computing model (ARM's big.LITTLE, but heavily

Message DeepSeek

🗨️ DeepThink 🔍 Search

📎

⬆️

Avinash Jeeva

...

AI-generated, for reference only

AI model 2: Qwen 3.8 Max

Current Ranking: 13

Search

36K

Try AI

Models

Pricing

Docs

Get Started

Model Playgrou...

Text Generation

Image Generation

Video Generation

Text to Speech

Speech to Text

← What is an expert System

+ Sign in for more models

API Call API

Qwen3.8-Max

What is an expert System

Qwen3.8-Max

An **Expert System** is a computer program that emulates the decision-making ability of a human expert in a specific, narrow domain. It is one of the earliest and most successful branches of Artificial Intelligence (AI).

Unlike modern AI (like Large Language Models) which relies on statistical probability and vast amounts of unstructured data, expert systems rely on **explicit, hand-coded knowledge** and **logical rules** to solve complex problems.

Core Architecture

An expert system typically consists of three main components:

1. **Knowledge Base:** The core repository containing domain-specific facts and heuristics (rules of thumb). This is usually structured as "IF-THEN" rules (e.g., *IF* symptom A AND symptom B are present, *THEN* diagnosis is likely C).

2. **Inference Engine:** The "brain" of the system. It applies logical rules to the knowledge base to deduce new information or reach conclusions. It can use:

- Forward Chaining:** Starts with known facts and works forward to a conclusion.
- Backward Chaining:** Starts with a hypothesis/goal and works backward to find supporting evidence.

3. **User Interface:** Allows non-experts to query the system and receive explanations for its reasoning.

Key Characteristics

- Domain-Specific:** They are highly accurate within their narrow field but useless outside of it.
- Explainability:** Unlike "black box" neural networks, expert systems can explain *why* they reached a specific conclusion by tracing the rule chain.

Type your message

+ Thinking Web Search

Sign in to claim 1M tokens of free inference quota

AI responses may be inaccurate—verify them before you rely on them. You hereby agree to the Qwen Cloud Website Customer Agreement, Privacy Policy, and Terms of Use.

Exercise 3

Selected AI Report: **Grok**

Report

DeepSeek

Using Generative AI Effectively and Responsibly in Probability and Statistics, Database Management Systems, and Software Engineering

Introduction

Generative AI tools such as ChatGPT, Claude, Gemini, and similar large language models are becoming widely available to university students. When used thoughtfully, they can act as on-demand tutors, practice partners, and study planners. When used carelessly, however, they can produce incorrect information, encourage shallow learning, and create academic integrity problems.

This report explains how a university student can use generative AI effectively and responsibly in three subjects: Probability and Statistics, Database Management Systems, and Software Engineering. For each subject, the report covers five areas: understanding concepts, practising problems, identifying mistakes, creating study plans, and preparing for examinations. It also discusses advantages, limitations, risks of over-reliance, and practical examples.

A central principle applies throughout: generative AI should be used as a learning aid, not as a replacement for your own reasoning, textbooks, lectures, or practical experience.

1. Probability and Statistics

Probability and Statistics requires both conceptual understanding and computational skill. Students often struggle with abstract ideas such as randomness, distributions, hypothesis testing, and

Grok

Using Generative AI Effectively and Responsibly to Learn Probability and Statistics, Database Management Systems, and Software Engineering

A practical report for university students

Abstract

Generative AI can accelerate learning in core computing subjects if it is treated as a tutor, critic, and drill generator — not as a substitute for thinking. This report explains how a university student can use large language models to understand concepts, practise problems, diagnose mistakes, plan study, and prepare for examinations in Probability and Statistics, Database Management Systems (DBMS), and Software Engineering. For each subject it gives concrete methods, sample prompts, and typical failure modes of the tools themselves. It then sets out advantages, limitations, and the academic and cognitive risks of over-reliance, and closes with a reusable weekly workflow and an integrity checklist.

The governing principle is simple: **the student must remain the reasoner**. AI is useful when it forces explanation, comparison, and correction. It is harmful when it produces finished answers that are copied, trusted without checking, or used to skip the struggle in which understanding is actually formed.

1. Introduction

Probability and Statistics, DBMS, and Software Engineering sit at the centre of most computer science and information technology degrees. They are also unusually well suited — and unusually dangerous — as targets for generative AI.

Qwen 3.8

The screenshot displays the QwenCloud web interface. At the top, there's a search bar and navigation links for 'Try AI', 'Models', 'Pricing', 'Docs', and a 'Get Started' button. The main content area shows a generated report titled 'Using Generative AI to Learn Probability and Statistics, Database Management Systems, and Software Engineering' with the subtitle 'A Comprehensive Guide for University Students'. The report includes a 'Table of Contents' with sections 1 through 9, and the beginning of the '1. Introduction' section. The interface also features a sidebar with 'Model Playground' options like Text, Image, Video Generation, and Text-to-Speech. At the bottom, there's a chat input field with a 'Type your message' prompt and a 'Sign in to claim 1M tokens of free inference quota' button. A footer note states: 'AI responses may be inaccurate—verify them before you rely on them. You hereby agree to the Qwen Cloud Website Customer Agreement, Privacy Policy, and Terms of Use.'

My thoughts: Grok's Report structure is well structured and contains a neat report structure compared to the other two models, that is why I chose this version as this closely resembles the structure that we learned in the technical writing module

Exercise 4

```
In [3]: import os
import openai
import tiktoken
from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file

openai.api_key = os.environ['OPENAI_API_KEY']

In [4]: # Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are a helpful assistant"}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "Explain how black holes are formed"}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

Black holes are formed when a massive star runs out of fuel and collapses in on itself during a supernova explosion. As the star collapses, the outer layers are blasted into space while the core collapses under its own gravity. If the core has a mass greater than about three times that of the Sun, it will collapse into a black hole.

The core of the star becomes so dense that its gravity becomes incredibly strong, causing it to collapse to a point of infinite density called a singularity. This singularity is surrounded by an event horizon, which is the point of no return for anything that crosses it. Once an object or light crosses the event horizon, it can never escape the gravitational pull of the black hole.

Black holes come in different sizes, with stellar-mass black holes being formed from the collapse of massive stars, and supermassive black holes found at the centers of galaxies, with masses millions or even billions of times that of the Sun.

```

In [6]: # Define system prompt
system_prompt = {
    "role": "system",
    "content": """"You are an expert event planner.
You have extensive knowledge of planning birthday parties,
university events, batch outings, weddings, and social events.
Provide practical, creative, and well-organized advice."""""
}

# Define user prompt
user_prompt = {
    "role": "user",
    "content": """"I am planning a birthday party for one of my friends.
Help me plan the event. Give me suggestions for the venue,
food, decorations, activities, budget, guest management,
and a timeline for the event."""""
}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)

# Print the response
print(response['choices'][0]['message']['content'])

```

Planning a birthday party can be a lot of fun and a great way to show your friend how much you care about them. Here are some suggestions to help you plan a successful birthday party:

1. **Venue**: Consider your friend's preferences and the number of guests. You can host the party at your home, rent a venue, or book a private room at a restaurant. Outdoor locations like parks or gardens can also be great options for a casual and fun atmosphere.
2. **Food**: Plan a menu based on your friend's favorite foods, dietary restrictions, and the time of day you're hosting the party. You can choose from catering services, potluck-style meals, or hiring a food truck for a more unique experience.
3. **Decorations**: Choose a theme or color scheme for the party and decorate accordingly. Balloons, streamers, banners, and personalized items like photos or a custom cake can add a special touch to the venue.
4. **Activities**: Organize activities or games based on your friend's interests. You can have a karaoke session, trivia games, a DIY craft station, or hire a DJ for dancing. Photo booths or props for fun pictures can also be a hit.
5. **Budget**: Determine a budget based on the number of guests, venue costs, food, decorations, and entertainment. Consider splitting expenses with friends if you're planning a joint party.
6. **Guest management**: Create a guest list and send out invitations in advance. Keep track of RSVPs to ensure you have an accurate headcount for food and seating arrangements.
7. **Timeline**:
 - 4-6 weeks before: Decide on a date, venue, and theme. Create a guest list and send out invitations.
 - 2-3 weeks before: Finalize the menu, decorations, and activities. Confirm bookings with vendors.
 - 1 week before: Follow up with guests who haven't RSVP'd. Prepare a schedule for the event.
 - Day of the party: Set up decorations, food, and activities. Greet guests and ensure everyone is having a good time.

Remember, the most important thing is to make the party memorable and enjoyable for your friend. Have fun planning, and happy birthday to your friend!

```
In [8]: # Define system prompt
system_prompt = {
    "role": "system",
    "content": """"You are an expert university study assistant.
You help students understand difficult technical subjects
by explaining concepts clearly, providing examples,
creating study plans, and giving practice questions.
Always explain concepts step by step and use simple language
when appropriate."""""
}

# Define user prompt
user_prompt = {
    "role": "user",
    "content": """"I have an upcoming examination in Artificial
Intelligence and Machine Learning. Create a 7-day study plan
covering supervised learning, unsupervised learning,
classification, regression, model evaluation, and
overfitting. Include daily topics, revision activities,
and practice questions."""""
}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)

# Print the response
print(response['choices'][0]['message']['content'])
```

Sure! Here is a 7-day study plan covering the mentioned topics:

```
### Day 1: Introduction to AI and ML
- Topics: Overview of AI and ML, supervised vs unsupervised learning
- Revision: Key concepts and terminologies in AI and ML
- Practice Questions: Define supervised and unsupervised learning with examples

### Day 2: Supervised Learning
- Topics: Definition, algorithms (e.g., linear regression, decision trees), applications
- Revision: Steps involved in supervised learning process
- Practice Questions: Implement a simple linear regression model using Python

### Day 3: Unsupervised Learning
- Topics: Definition, algorithms (e.g., k-means clustering, PCA), applications
- Revision: Differences between supervised and unsupervised learning
- Practice Questions: Apply k-means clustering to a dataset

### Day 4: Classification
- Topics: Definition, algorithms (e.g., logistic regression, SVM, decision trees), evaluation metrics
- Revision: Understand confusion matrix, precision, recall, F1 score
- Practice Questions: Calculate precision, recall, and F1 score for a classification problem

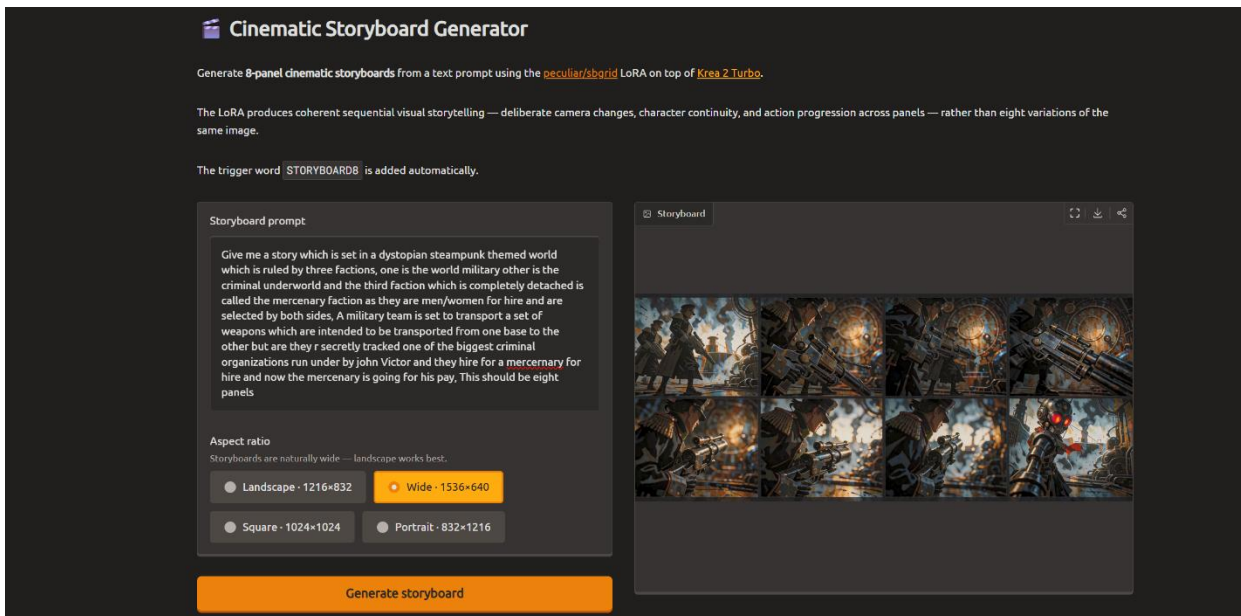
### Day 5: Regression
- Topics: Definition, algorithms (e.g., linear regression, polynomial regression), evaluation metrics
- Revision: Understand Mean Squared Error (MSE), R-squared
- Practice Questions: Evaluate a regression model using MSE and R-squared

### Day 6: Model Evaluation and Overfitting
- Topics: Cross-validation, Overfitting, Bias-Variance tradeoff
- Revision: Techniques to prevent overfitting (e.g., regularization)
- Practice Questions: Explain Bias-Variance tradeoff with examples

### Day 7: Revision and Practice
- Review all topics covered during the week
- Solve mock exams or practice questions from past papers
- Identify weak areas for further revision
```

Remember to dedicate time each day for revision and practice to reinforce your understanding of the concepts. Good luck with your examination!

Exercise 5



Identified model

```
(1) | pip install -q "transformers<5.0.0"
✓ 2s

(2) import transformers
print(transformers.__version__)
4.37.6

(3) from transformers import pipeline
summarizer = pipeline("summarization")
summary = summarizer(
    """Hugging Face is a pioneering open-source AI platform
    that empowers developers, researchers, and organizations to easily access and
    deploy cutting-edge machine learning models for natural language processing,
    computer vision, and beyond—democratizing AI and accelerating innovation
    through a collaborative ecosystem of tools, datasets, and community-driven
    contributions.""",
    max_length=20
)
print(summary)

No model was supplied, defaulted to sshleifer/distilbart-cnn-12-6 and revision a48f3e (https://huggingface.co/sshleifer/distilbart-cnn-12-6).
Using a pipeline without specifying a model name and revision in production is not recommended.
/usr/local/lib/python3.13/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret "HF_TOKEN" does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 1.00k/7 [00:00-00:00, 140kB/s]
pytorch_model.bin: 100% 1.22G/1.22G [00:13-00:00, 92.1MB/s]
model.safetensors: 67% 814M/1.22G [00:04-00:01, 262MB/s]
tokenizer_config.json: 100% 26.0/26.0 [00:00-00:00, 5650B/s]
vocab.json: 896k/7 [00:00-00:00, 11.1MB/s]
merges.txt: 450k/7 [00:00-00:00, 5.47MB/s]
Device set to use cuda:0
Your min_length=56 must be inferior than your max_length=20.
/usr/local/lib/python3.13/dist-packages/transformers/generation/utils.py:1633: UserWarning: Unfeasible length constraints: 'min_length' (56) is larger than the maximum possible length (20). Generation will stop at the defined maximum length. You should
warnings.warn(
[{"summary_text": "Hugging Face is a pioneering open-source AI platform that empowers developers, researchers"}]]
```

```
[4] 16 from transformers import pipeline

# Use a known summarization model
summarizer = pipeline("summarization", model="thunus/kobart-summary-v3")

# Input text to summarize
text = """Hugging Face is a pioneering open-source AI platform
that empowers developers, researchers, and organizations to easily access and
deploy cutting-edge machine learning models for natural language processing, computer vision,
and beyond—democratizing AI and accelerating innovation through a collaborative ecosystem of tools,
datasets, and community-driven contributions."""

# Generate summary
summary = summarizer(text, max_length=20, min_length=5, do_sample=False)
print(summary)

... config.json 100% 168K/9 [00:00<00:00, 157KB/s]
You passed 'num_labels=3' which is incompatible to the 'id2label' map of length '2'.
You passed 'num_labels=3' which is incompatible to the 'id2label' map of length '2'.
model.safetensors 100% 490M/490M [00:11<00:00, 44.9MB/s]
generation_config.json 100% 191/191 [00:00<00:00, 10.5KB/s]
tokenizer_config.json 39.5K/7 [00:00<00:00, 1.65MB/s]
tokenizer.json 1.05M/7 [00:00<00:00, 36.8MB/s]
special_tokens_map.json 100% 602/602 [00:00<00:00, 66.9KB/s]
Device set to use cuda:0
Both 'max_new_tokens' (-256) and 'max_length' (-20) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
[('summary_text': 'Hugging Face is a pioneering open-source AI platform that empowers developers, researchers, and organizations to easily access and deploy cutting-edge machine learning models for natural language processing, computer vision, and bey
```

```
import requests
from PIL import Image

from transformers import AutoImageProcessor, AutoModel

url = "http://images.cocodataset.org/val2017/000000039769.jpg"
image = Image.open(requests.get(url, stream=True).raw)

processor = AutoImageProcessor.from_pretrained("apple/aimv2-large-patch14-native")
model = AutoModel.from_pretrained("apple/aimv2-large-patch14-native", device_map="auto")

inputs = processor(images=image, return_tensors="pt").to(model.device)
outputs = model(**inputs)

... preprocessor_config.json 100% 638/638 [00:00<00:00, 26.7KB/s]
config.json 100% 768/768 [00:00<00:00, 24.5KB/s]
model.safetensors 100% 1.24G/1.24G [00:06<00:00, 258MB/s]
```

```
[7] 46 import requests
import torch
from PIL import Image
from transformers import AutoModel, AutoProcessor

# Download the image
url = "http://images.cocodataset.org/val2017/000000039769.jpg"
image = Image.open(requests.get(url, stream=True).raw)

# Text descriptions to compare with the image
text = [
    "Picture of a dog.",
    "Picture of a cat.",
    "Picture of a horse."
]

# Load the processor
processor = AutoProcessor.from_pretrained(
    "apple/aimv2-large-patch14-224-lit"
)

# Load the model
device = "cuda" if torch.cuda.is_available() else "cpu"
model = AutoModel.from_pretrained(
    "apple/aimv2-large-patch14-224-lit"
).to(device)

# Prepare inputs
inputs = processor(
    images=image,
    text=text,
    add_special_tokens=True,
    truncation=True,
    padding=True,
    return_tensors="pt"
)

# Move all input tensors to the same device as the model
inputs = {
    key: value.to(device)
    for key, value in inputs.items()
    if hasattr(value, "to")
}

# Run the model
with torch.no_grad():
    outputs = model(**inputs)

# Calculate probabilities
probs = outputs.logits_per_image.softmax(dim=-1)

print("Device:", device)
print("Probabilities:", probs)

... Device: cuda
Probabilities: tensor([[1.4047e-03, 9.9859e-01, 9.5533e-06]], device='cuda:0')
```